Project 2

Autonomous Vehicle: Putting it All Together

COMPENG 4DS4 - Winter 2026

McMaster University

Group 19

Ali Raza - 400338538

Zuhaib Quraishi - 400306494

Khushi Akbari - 400320925

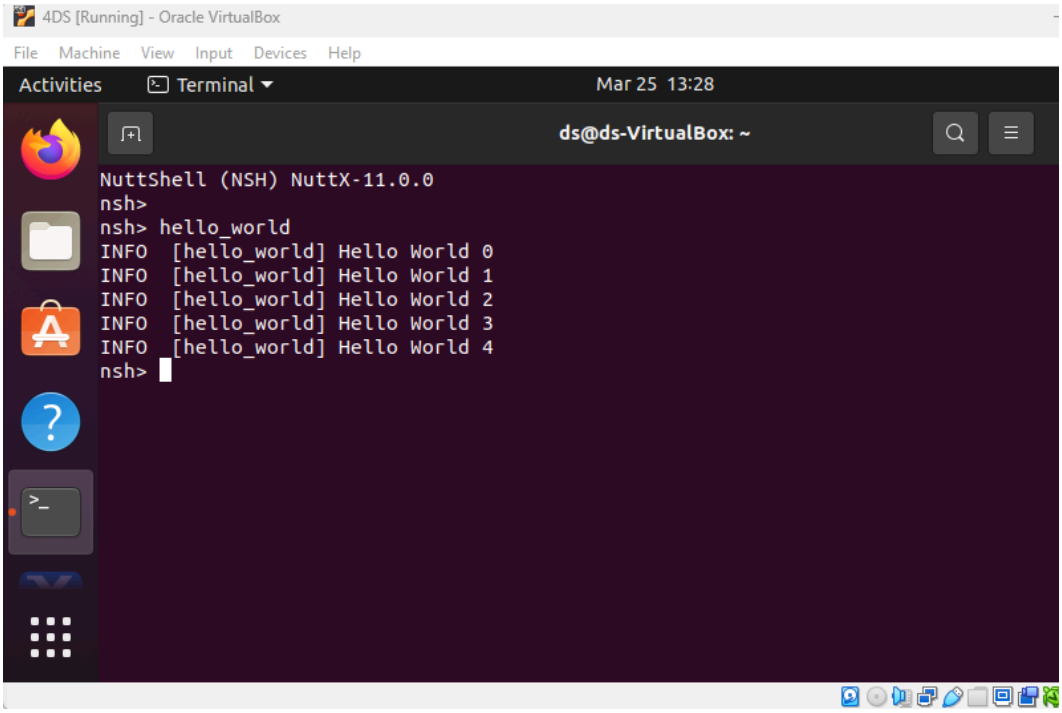
Declaration of Contributions

Ali Raza: Experiment 1 (Build and program PX4). Experiment 2 (hello_world application on NuttX). Experiment 3A (uORB messaging, subscribe and publish). Project 2 Part 1 (RC controller motor and servo control). Project 2 Part 2 (autonomous control via MAVLink).

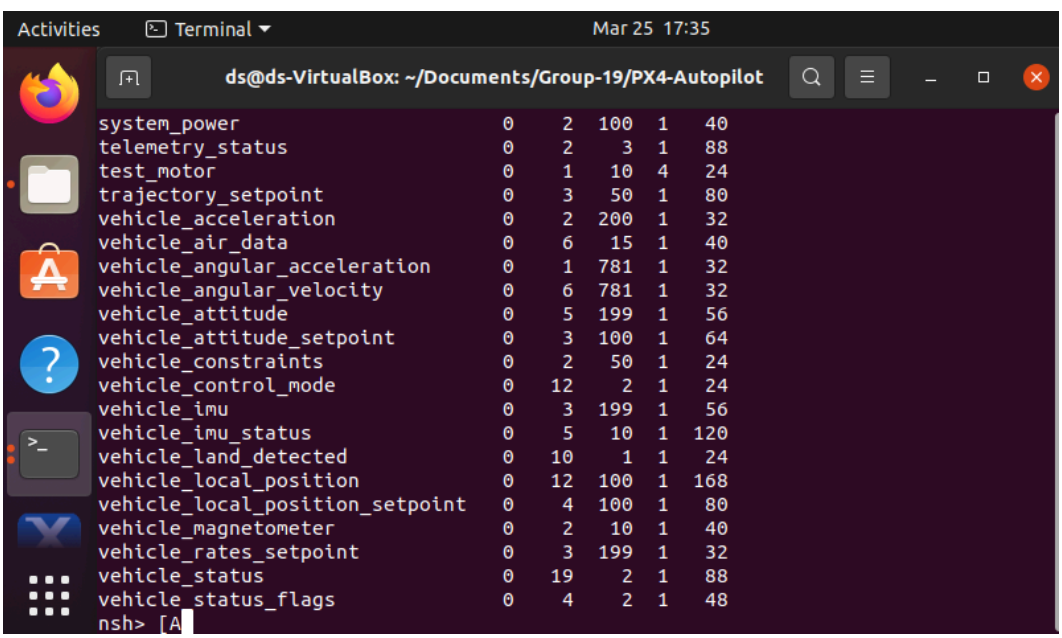
Zuhaib Quraishi: Experiment 3C (motor control via test_motor). Experiment 4 (MAVLink setup and communication). Project 2 Part 1 (RC controller motor and servo control). Project 2 Part 2 (autonomous control via MAVLink).

Khushi Akbari: Experiment 5 (ultrasonic sensor on Raspberry Pi). Experiment 6 (camera direction detection). Project 2 Part 1 (RC controller motor and servo control). Project 2 Part 2 (autonomous control via MAVLink).

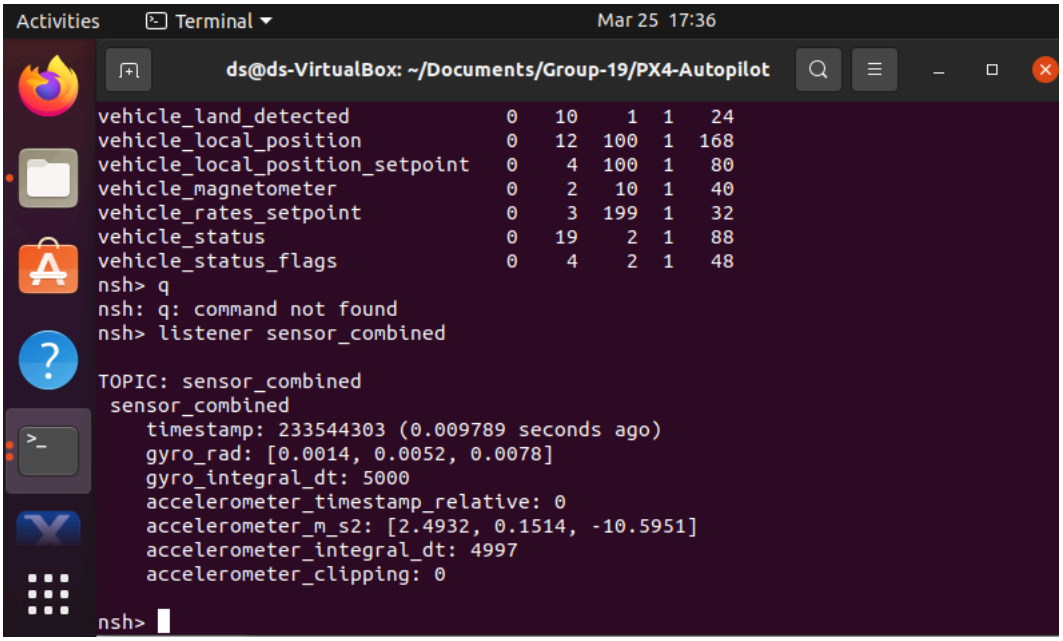
Experiment 2B: Hello World Application



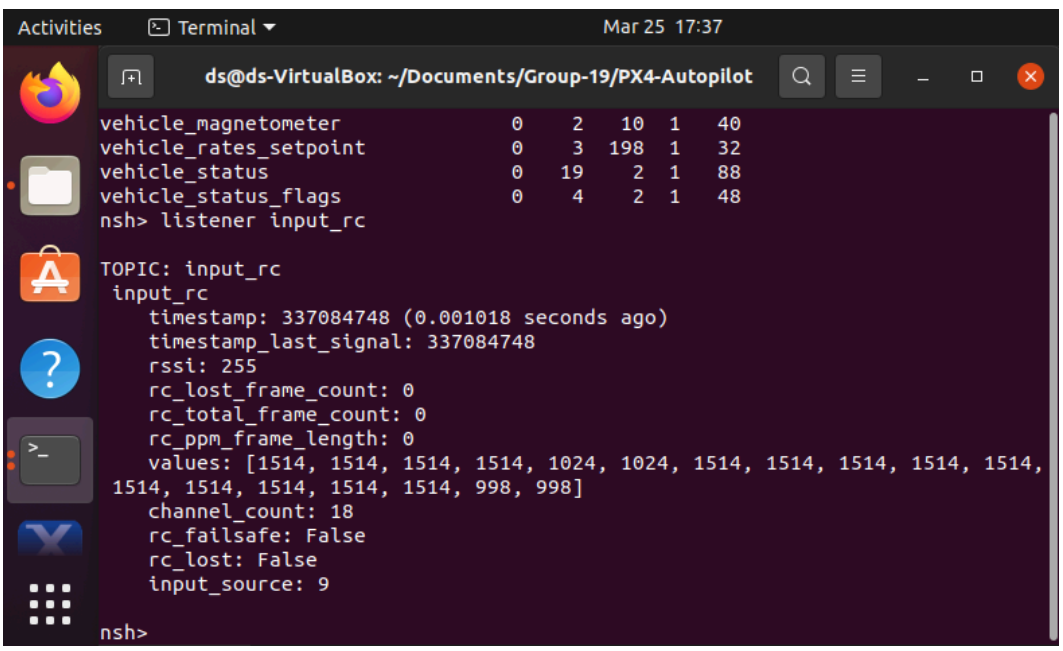
Experiment 3A: uORB Top



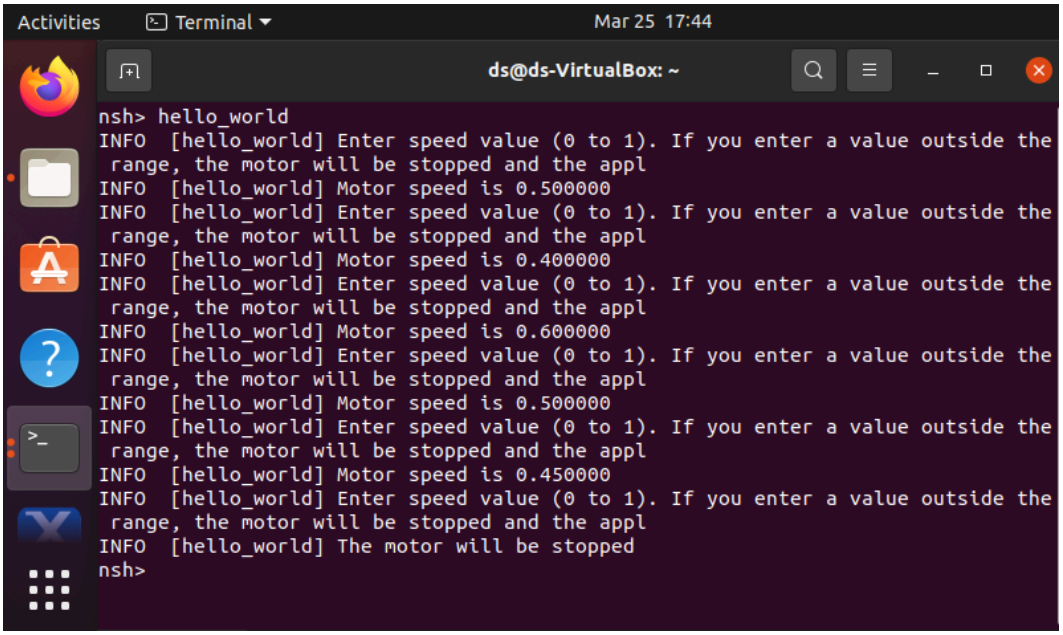
Experiment 3A: Listener sensor_combined



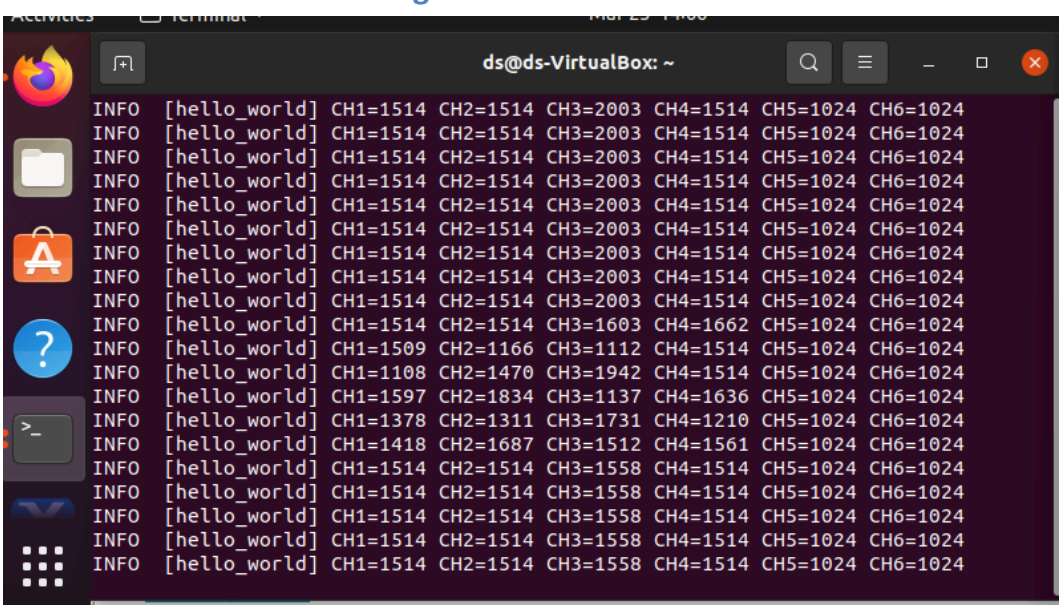
Experiment 3A: Listener input_rc



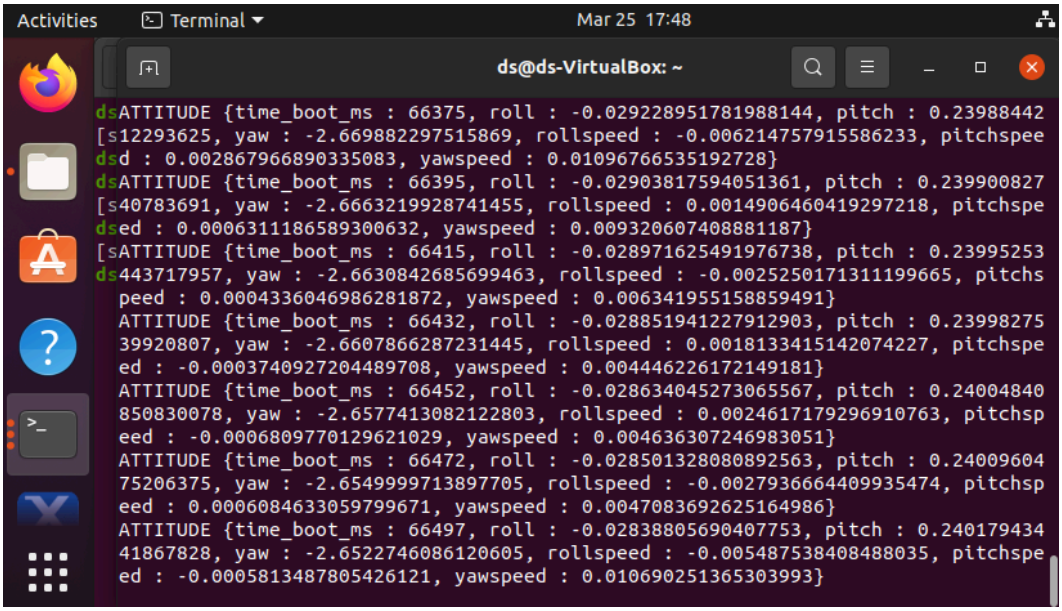
Experiment 3C: Motor Speed Control



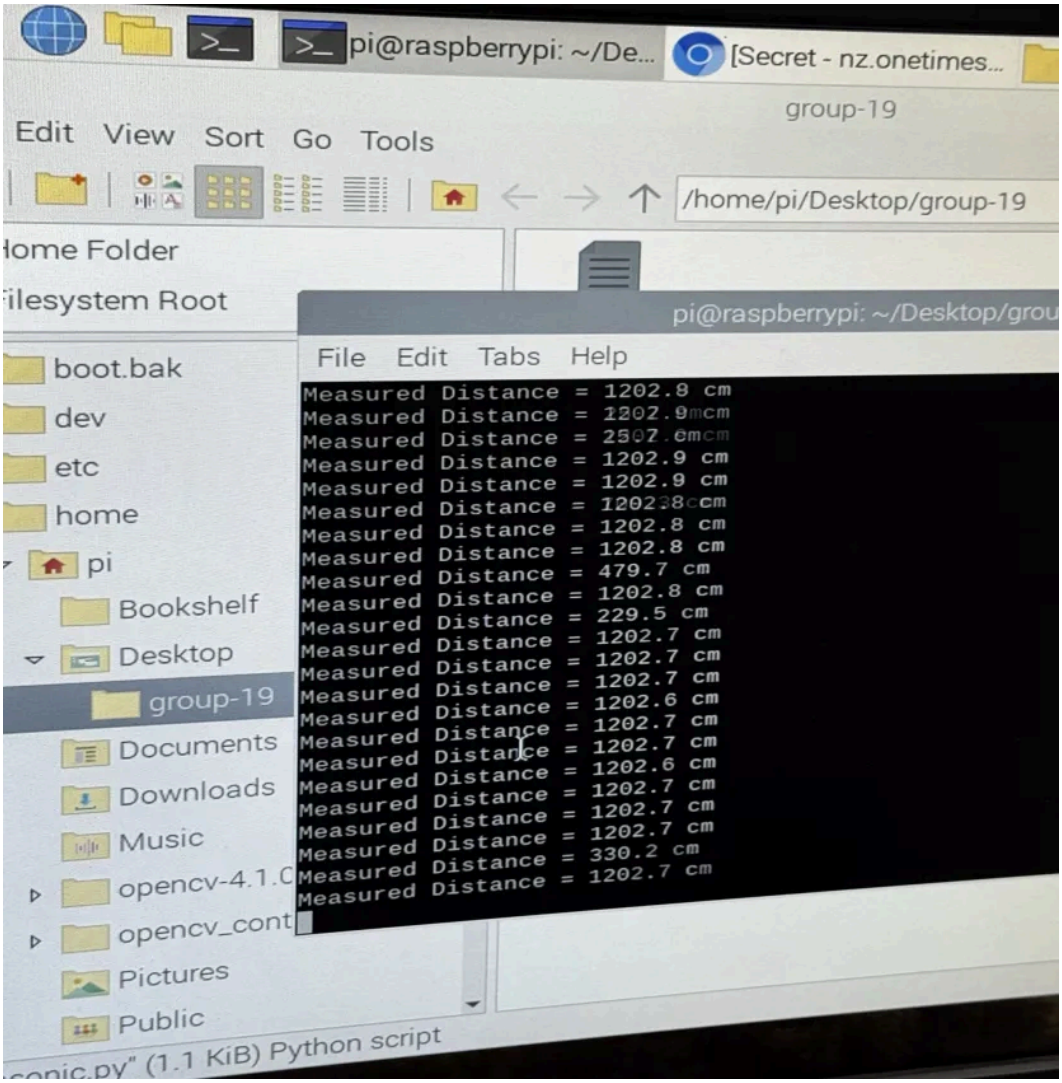
Experiment 4A: MAVLink Message Stream



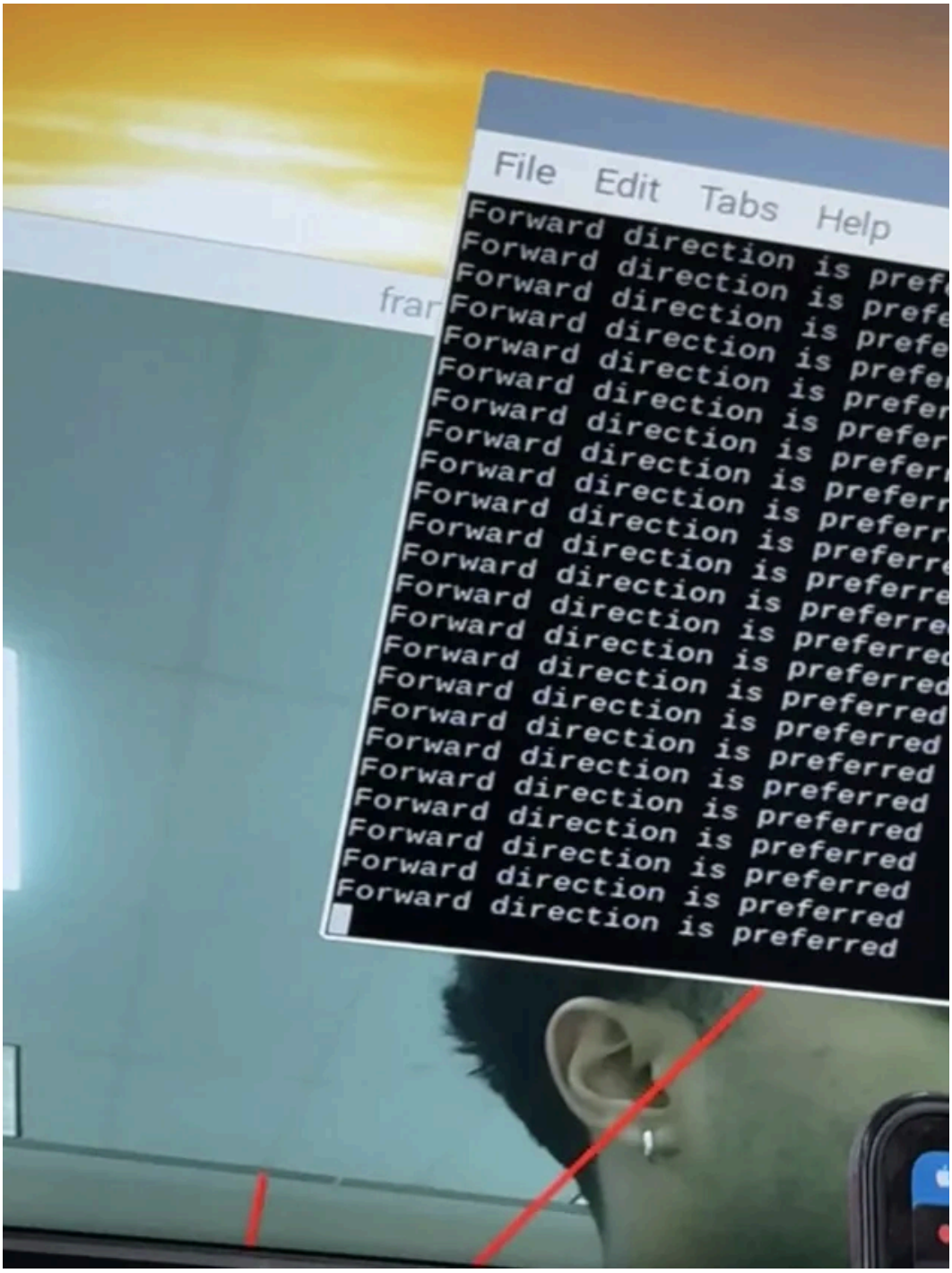
Experiment 4A: ATTITUDE Messages



Experiment 5: Ultrasonic Distance Readings



Experiment 6: Camera Direction Detection



P2 PART 1: hello.cpp

```
#include <px4_platform_common/px4_config.h>

#include <px4_platform_common/log.h>
```

```
#include <drivers/drv_hrt.h>
#include <uORB/Subscription.hpp>
#include <uORB/Publication.hpp>
#include <uORB/topics/input_rc.h>
#include <uORB/topics/test_motor.h>

// FMU output pins (0-based, so pin 1 = 0)
#define DC_MOTOR        0 //connected to output pin 1
#define SERVO           1 //connected to output pin 2

// FlySky RC channels (0-based)
#define RC_THROTTLE_CH  2   // CH3: left stick vertical, array index 2
#define RC_STEERING_CH  0   // CH1: right stick horizontal, array index 0

// RC signal range in microseconds
#define RC_MIN  1000
#define RC_MAX  2000

extern "C" __EXPORT int hello_world_main(int argc, char *argv[]);

// converts RC value (1000-2000) to a float between out_min and out_max
static float rc_map(uint16_t rc_val, float out_min, float out_max)
{
    //(rc_value -1000)/ (2000-1000)
    float t = (float)(rc_val - RC_MIN) / (float)(RC_MAX - RC_MIN);
    //clamp
    if (t < 0.0f) t = 0.0f;
    if (t > 1.0f) t = 1.0f;
    return out_min + t * (out_max - out_min);
}
```

```
int hello_world_main(int argc, char *argv[])
{
    //create publisher for test_motor & subscriber to the input_rc
    uORB::Publication<test_motor_s> motor_pub{ORB_ID(test_motor)};
    uORB::Subscription rc_sub{ORB_ID(input_rc)};

    input_rc_s   rc_data{}; //stores latest RC input message
    test_motor_s motor_cmd{}; //hold DC motor command before publishing
    test_motor_s servo_cmd{}; //hold servo command before publishing

    PX4_INFO("RC motor/servo control started.");
    PX4_INFO("Throttle -> CH%d | Steering -> CH%d", RC_THROTTLE_CH + 1, RC_STEERING_CH +
1);
    PX4_INFO("Make sure RC receiver is connected and RC controller is on.");

    while (1)
    {
        if (rc_sub.update(&rc_data)) // new RC message has arrived
        {
            if (rc_data.rc_lost)
            {
                PX4_WARN("RC signal lost - holding last command");
            }
            else
            {
                // throttle: 1000=full reverse, 1500=stop, 2000=full forward
                float motor_val = rc_map(rc_data.values[RC_THROTTLE_CH], 0.0f, 1.0f);

                // steering: 1000=full left, 1500=center, 2000=full right
                float servo_val = rc_map(rc_data.values[RC_STEERING_CH], 0.0f, 1.0f);
```

```
        //after rc_map() computes motor_val and servo_val, they are written
        //into a test_motor_s struct and published

        // send motor speed
        motor_cmd.timestamp       = hrt_absolute_time();
        motor_cmd.motor_number    = DC_MOTOR;
        motor_cmd.value           = motor_val;
        motor_cmd.action          = test_motor_s::ACTION_RUN;
        motor_cmd.driver_instance = 0;
        motor_cmd.timeout_ms      = 0;
        motor_pub.publish(motor_cmd);

        // send servo position (same topic, different motor_number)
        servo_cmd.timestamp       = hrt_absolute_time();
        servo_cmd.motor_number    = SERVO;
        servo_cmd.value           = servo_val;
        servo_cmd.action          = test_motor_s::ACTION_RUN;
        servo_cmd.driver_instance = 0;
        servo_cmd.timeout_ms      = 0;
        motor_pub.publish(servo_cmd);

        PX4_INFO("Motor: %.2f  Servo: %.2f  (raw: throttle=%d steering=%d)",
                (double)motor_val, (double)servo_val,
                rc_data.values[RC_THROTTLE_CH],
                rc_data.values[RC_STEERING_CH]);
    }
}

px4_usleep(200000); // 5 Hz, sleeps for 0.2 seconds, loop runs about 5 times per second
```

```
    }

    // stop everything on exit
    PX4_INFO("Stopping motor and centering servo");

    motor_cmd.timestamp       = hrt_absolute_time();
    motor_cmd.motor_number    = DC_MOTOR;
    motor_cmd.value           = 0.5f;
    motor_cmd.action          = test_motor_s::ACTION_RUN;
    motor_cmd.driver_instance = 0;
    motor_cmd.timeout_ms      = 0;
    motor_pub.publish(motor_cmd);

    servo_cmd.timestamp       = hrt_absolute_time();
    servo_cmd.motor_number    = SERVO;
    servo_cmd.value           = 0.5f;
    servo_cmd.action          = test_motor_s::ACTION_RUN;
    servo_cmd.driver_instance = 0;
    servo_cmd.timeout_ms      = 0;
    motor_pub.publish(servo_cmd);

    return 0;
}
```

P2 PART 2: hello.cpp

```
#include <px4_platform_common/px4_config.h>
#include <px4_platform_common/log.h>
#include <drivers/drv_hrt.h>
#include <uORB/Publication.hpp>
#include <uORB/topics/test_motor.h>
```

```
#include <uORB/topics/debug_value.h>

// FMU output pins (0-based, so pin 1 = 0)
#define DC_MOTOR        0 //output pin 1
#define SERVO           1 //output pin 2

// motor: 0.5 = stop, above 0.5 = forward
#define MOTOR_STOP      0.5f
#define MOTOR_SLOW      0.58f
#define MOTOR_NORMAL    0.65f

// servo: 0.5 = center, 0.25 = left, 0.75 = right
#define SERVO_LEFT      0.25f
#define SERVO_CENTER    0.5f
#define SERVO_RIGHT     0.75f

// distance thresholds from ultrasonic (cm)
#define DIST_STOP       15.0f //15cm, stop
#define DIST_SLOW       50.0f //50cm, slowdown

extern "C" __EXPORT int hello_world_main(int argc, char *argv[]);

// helper to publish a test_motor command without repeating all the fields
static void publish_motor(uORB::Publication<test_motor_s> &pub, uint32_t motor_num, float
value)
{
    test_motor_s msg{};
    msg.timestamp       = hrt_absolute_time();
    msg.motor_number    = motor_num; //which output to control
    msg.value           = value; //command value to send
    msg.action          = test_motor_s::ACTION_RUN;
```

```
    msg.driver_instance = 0;
    msg.timeout_ms      = 0;
    pub.publish(msg);
}

int hello_world_main(int argc, char *argv[])
{
    // wait for MAVLink and uORB to finish starting up
    px4_sleep(2);

    // subscribe to debug_value - this is what MAVLink maps incoming debug messages to
    debug_value_s debug_data{};
    int debug_handle = orb_subscribe(ORB_ID(debug_value));
    orb_set_interval(debug_handle, 100); // 1000/100 = 10 Hz, ask for updates 10 times per seocnd

    //create publisher for the test_motor topic
    uORB::Publication<test_motor_s> motor_pub{ORB_ID(test_motor)};

    // start safe
    publish_motor(motor_pub, DC_MOTOR, MOTOR_STOP);
    publish_motor(motor_pub, SERVO,    SERVO_CENTER);

    while (1)
    {
        //copies the latest debug_value message into debug_data
        //after this point debug_data contains latest info sent from raspberry pi
        orb_copy(ORB_ID(debug_value), debug_handle, &debug_data);

        // RPi sends: ind = direction (0=left, 1=forward, 2=right), value = distance in cm
        int   direction = (int)debug_data.ind; //direction from camera
```

```
    float dist_cm   = debug_data.value; //distrance from ultrasonic

    // slow down or stop based on distance
    float motor_val;
    if (dist_cm < DIST_STOP)
        motor_val = MOTOR_STOP;
    else if (dist_cm < DIST_SLOW)
        motor_val = MOTOR_SLOW;
    else
        motor_val = MOTOR_NORMAL;

    // steer based on camera direction
    float servo_val;
    if (direction == 0)
        servo_val = SERVO_LEFT;
    else if (direction == 2)
        servo_val = SERVO_RIGHT;
    else
        servo_val = SERVO_CENTER;

    //publish commands
    publish_motor(motor_pub, DC_MOTOR, motor_val);
    publish_motor(motor_pub, SERVO,    servo_val);

    PX4_INFO("dist: %.1f cm | dir: %d | motor: %.2f | servo: %.2f",
            (double)dist_cm, direction, (double)motor_val, (double)servo_val);

    px4_usleep(100000); // loop runs at about 10 Hz
}
```

```
    return 0;
}
```

P2 PART 2: main.py

```
#runs on the raspberry pi, its job is to look with the camera, listen with the ultrasonic sensor
#make decisions and send the into to FMU
import RPi.GPIO as GPIO
import cv2
import numpy as np
import math
import time
from pymavlink import mavutil

# ultrasonic sensor pins
GPIO_TRIGGER = 23
GPIO_ECHO = 24

# open camera
cap = cv2.VideoCapture(0)
StepSize = 5 #when scanning image check every 5 columns

# set up GPIO pins for ultrasonic
def ultrasonic_setup():
    GPIO.setmode(GPIO.BCM)
    GPIO.setup(GPIO_TRIGGER, GPIO.OUT)
    GPIO.setup(GPIO_ECHO, GPIO.IN)

# fire the sensor and return distance in cm
def get_distance():
    # send a 10us pulse to trigger
    GPIO.output(GPIO_TRIGGER, True)
```

```
    time.sleep(0.00001)
    GPIO.output(GPIO_TRIGGER, False)

    # 30ms timeout = ~5m max range, returns 999 if no echo
    timeout = 0.03
    loop_start = time.time()

    # wait for echo to go high (sound left the sensor)
    StartTime = time.time()
    while GPIO.input(GPIO_ECHO) == 0:
        StartTime = time.time()
        if time.time() - loop_start > timeout:
            return 999.0

    # wait for echo to go low (sound came back)
    StopTime = time.time()
    loop_start = time.time()
    while GPIO.input(GPIO_ECHO) == 1:
        StopTime = time.time()
        if time.time() - loop_start > timeout:
            return 999.0

    # distance = (time * speed of sound) / 2 (divide by 2 because there and back)
    TimeElapsed = StopTime - StartTime
    distance = (TimeElapsed * 34300) / 2
    return distance

# splits a list into chunks of size n
    #used later to split camera edge points into left, middle and right section
def getChunks(l, n):
```

```
    """Yield successive n-sized chunks from l."""
    a = []
    for i in range(0, len(l), n):
        a.append(l[i:i + n])
    return a

# reads a frame and returns preferred direction: 0=left, 1=forward, 2=right
def get_direction():
    ret, frame = cap.read() # read one frame from camera
    if not ret:
        return None

    # flip, filter noise, detect edges
    img = cv2.flip(frame, 0) #flip image vertically
    blur = cv2.bilateralFilter(img, 9, 40, 40)
    edges = cv2.Canny(blur, 50, 100)
    img_h = img.shape[0] - 1
    img_w = img.shape[1] - 1

    # scan each column from bottom up, grab the first edge pixel found
    #objects closer to the car show uo lower in the image
    EdgeArray = []
    for j in range(0, img_w, StepSize):
        pixel = (j, 0)
        for i in range(img_h - 5, 0, -1):
            if edges.item(i, j) == 255:
                pixel = (j, i)
                EdgeArray.append(pixel)
                break
```

```
if len(EdgeArray) == 0: #no edges found
    return None

# split edge pixels into 3 vertical chunks (left, center, right)
chunks = getChunks(EdgeArray, math.ceil(len(EdgeArray) / 3))

#c = []
distance = []
for i in range(len(chunks)): #for each chunk 1 find everage point
    x_vals = []
    y_vals = []
    for (x, y) in chunks[i]:
        x_vals.append(x)
        y_vals.append(y)
    avg_x = int(np.average(x_vals))
    avg_y = int(np.average(y_vals))
    #c.append([avg_y, avg_x])

    # distance from bottom-center of frame to mean edge point of this chunk
    distance.append(math.sqrt((avg_x - 320) ** 2 + (avg_y - 640) ** 2))
    cv2.line(img, (320, 640), (avg_x, avg_y), (0, 0, 255), 2)

cv2.imshow("frame", img)
cv2.waitKey(5)

if len(distance) < 3: #if there are not 3 proper chunks return none
    return None

# shortest distance = least obstruction = preferred direction
if distance[0] < distance[1]:
```

```
        if distance[0] < distance[2]:
            return 0  # left
        else:
            return 2  # right
    else:
        if distance[1] < distance[2]:
            return 1  # forward
        else:
            return 2  # right

# setup ultrasonic
ultrasonic_setup()

# connect to FMU over USB, baud must match mavlink start command on FMU
the_connection = mavutil.mavlink_connection('/dev/ttyACM0', baud=2000000)

the_connection.wait_heartbeat() #wait until FMU sends a heartbeat
#print info showing connection worked
print("Heartbeat from system (system %u component %u)" % (the_connection.target_system,
      the_connection.target_component))

while True:
    dist = get_distance()

    direction = get_direction()
    if direction is None:
        direction = 1  # default to forward if camera gives nothing

    # pack both values into one MAVLink debug message
    # ind = direction, value = distance in cm
```

```
message = mavutil.mavlink.MAVLink_debug_message(0, direction, dist)
#sends to FMU
the_connection.mav.send(message)

print("Sent -> direction: %d  distance: %.1f cm" % (direction, dist))
time.sleep(0.1)  # 10 Hz
```